

MONITOR Z · L08

Eventos de verdad: ENF 70

Cuaderno de estudio · versión del 2026-10-05

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L08 — Eventos de verdad: ENF 70

Objetivo: pasar del nivel B (preguntar cada N segundos) al nivel C, en el que **JES avisa** cuando un job arranca o termina. Al final sabés cómo se escucha la señal, qué reglas tiene el exit y por qué el polling no se tira.

 **Requiere de sistemas:** que el programa venga de una **librería APF** y corra como **STC** (L07).

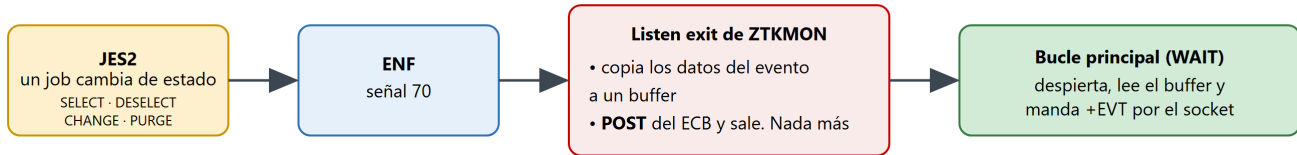
1. Qué es ENF

ENF (*Event Notification Facility*) es el mecanismo con el que z/OS avisa eventos a quien se haya anotado para escucharlos. Cada tipo de evento tiene un **código**. El **70** lo emite **JES2** cuando un job cambia de estado:

Evento	Qué significa para nosotros
SELECT	El job fue tomado para ejecutar → +EVT INI
DESELECT	Terminó de ejecutar, con el motivo (normal, CC,abend, JCL, cancelado, flush) → +EVT FIN
CHANGE	Cambió algún atributo o fase → +EVT FASE (opcional)
PURGE	Se purgó → +EVT PURGA (opcional)

Los datos del evento se leen con el mapeo **IAZENF70** de SYS1.MACLIB.

 Fuente: *z/OS MVS Programming: Authorized Assembler Services Reference* (ENFREQ) · mapeo IAZENF70 · IDEA-005, nivel C.



Al arrancar: ENFREQ ACTION=LISTEN, CODE=70, EXIT=... · al parar (P ZTKMON): ENFREQ ACTION=DELETE
 Los datos del evento se leen con el mapeo IAZENF70 (SYS1.MACLIB).

🔔 Analogía: el timbre del edificio (nivel C de L00)

En vez de que el vigilante haga rondas, JES **toca el timbre** cada vez que un job entra o sale. El exit es el **timbre**: sólo suena y deja una nota; no puede ponerse a conversar. El que atiende es el bucle principal.

Fuente: z/OS MVS Programming: Authorized Assembler Services Reference (ENFREQ) · mapeo IAZENF70 · IDEA-005, nivel C

Lámina L08-01_flujo_enf70

2. Anotarse y borrarse

Momento	Llamada	Qué hace
Al arrancar	ENFREQ ACTION=LISTEN, CODE=70, EXIT=...	Registra tu exit para la señal 70 y devuelve un token de la escucha
Al parar (P ZTKMON)	ENFREQ ACTION=DELETE con ese token	Deja de escuchar

Si no hacés el DELETE, la señal sigue apuntando a un exit que ya no existe. Por eso el DELETE va también en la rutina de recuperación (L07).

3. La autorización

ENFREQ ACTION=LISTEN es un servicio **autorizado**. El que lo llama tiene que estar en **estado supervisor** o en **clave PSW 0–7**.

- Un programa de una **librería APF**, enlazado con AC=1, puede pasar a estado supervisor con MODESET, hacer el pedido y **volver enseguida** a estado problema.
- **Principio:** estar autorizado sólo el tiempo mínimo. Cuanto menos código corre autorizado, menos daño puede hacer un error.

4. El listen exit: reglas de oro

Regla	Por qué
Corto: copiar los datos y hacer POST	Corre en el contexto del que emite la señal: si se demora, demora a JES
Sin I/O ni esperas	Nada de WTO largos, PUT ni sockets: eso lo hace el bucle principal
Pensado para ráfagas	En el pico del cierre llegan muchos eventos seguidos
Dónde tiene que residir el exit y en qué modo corre	A confirmar en el manual de ENFREQ antes de escribirlo (requisitos de memoria del exit)

5. El buffer entre el exit y el bucle

El exit y el bucle principal **corren a la vez**, así que comparten un buffer con cuidado:

- Un **buffer circular** de N entradas, con dos índices: **escribir** (lo mueve el exit) y **leer** (lo mueve el bucle).
- El exit avanza su índice con **CS** (*compare and swap*), una instrucción atómica. Así dos eventos simultáneos no pisan la misma entrada.
- **Si el buffer se llena**, el exit no espera: marca una bandera de "desborde" y descarta. Al ver la bandera, el bucle **reconcilia con SSI 80** (**ACT** + **POSTEX**) y nada se pierde de verdad.

Criterio propio. Es el diseño a validar en la práctica; el tamaño del buffer se elige con la ráfaga de L04 (50 jobs).

6. Nivel B vs. nivel C

	B · SSI 80 cada N s	C · ENF 70
Latencia	Hasta N segundos	Inmediata
CPU sin actividad	Una consulta por vuelta	≈ 0
Permisos	Ninguno	APF + STC
Si se pierde un evento	La próxima vuelta lo ve	Hay que reconciliar con SSI 80

Por eso B no se tira: en el nivel C, SSI 80 sigue sirviendo para reconciliar al arrancar, después de un desborde y en cada reconexión (ACT, HIST).

Las reglas del listen exit y de la autorización			L08 · Eventos de verdad: ENF 70
Regla		Por qué	
El exit es corto: copiar y POST		Corre en el contexto del que emite la señal: si se demora, demora a JES	
Sin I/O ni esperas en el exit		Nada de WTO largos, PUT ni sockets: eso lo hace el bucle	
El buffer entre exit y bucle debe tolerar ráfagas		En el pico del cierre pueden llegar muchos eventos seguidos; hay que decidir qué pasa si se llena	
ENFREQ LISTEN exige estar autorizado		Estado supervisor o clave 0-7: el programa tiene que venir de una librería APF	
Dónde debe residir el exit		A confirmar en el manual al llegar a L08 (requisitos de memoria del exit para ENFREQ)	
Al terminar: ENFREQ ACTION=DELETE		Si no, la señal sigue apuntando a un exit que ya no existe	
Comparación		Nivel B (SSI 80 cada N s)	Nivel C (ENF 70)
Latencia		Hasta N segundos	Inmediata
CPU sin actividad		Una consulta por vuelta	≈ 0
Permisos		Ninguno	APF + STC
Si se pierde un evento		La próxima vuelta lo ve	Hace falta reconciliar con SSI 80 (ACT / HIST)
Por eso B no se tira: en el nivel C, SSI 80 sigue sirviendo para reconciliar al arrancar y después de cualquier corte.			
Fuente: Authorized Assembler Services Reference (ENFREQ) · criterio propio para la comparación (se mide en la práctica)			

Lámina L08-02_reglas_del_exit

7. Preguntas de comprobación

1. ¿Por qué el exit no puede escribir directamente en el socket?
2. ¿Qué pasa si dos eventos llegan al mismo tiempo y el exit avanza el índice con L / A / ST en vez de CS ?
3. ¿Qué hacés si el buffer se llena en el pico del cierre?
4. ¿Por qué hay que hacer ENFREQ DELETE también en la rutina de recuperación?
5. Con el nivel C funcionando, ¿para qué seguís necesitando SSI 80?

